Technical CV Analysis & Learning Guide

Prepared for Najat El Otmani — target: Data Science / ML / AI Engineering / Data Engineering / MLOps PFE (6 months, from January 2027). Bilingual 🇬🇧 / 🇫🇷.

Note on YouTube: I cannot open YouTube from here, so I do not invent video URLs. For every topic I give the channel + exact search query + a ready search link. Open the first result from the named channel.

---

1. Executive Summary

🇬🇧 **How a recruiter reads your CV in 20 seconds:** a Telecom/Networks engineering student who moved into ML, with one genuinely interesting thread: **IoT sensor data → time-series forecasting → optimized model serving** (AirSentinel: Transformer → ONNX Runtime → FastAPI → Docker → Streamlit). That thread is your identity. The rest (Laravel, California housing, solar ARIMA) is supporting material.

Real strengths: deployment mindset (ONNX + FastAPI + Docker is rare for a student), time-series focus with a baseline comparison (persistence), honest numeric results, IoT/network background that most data students lack, certifications (Oracle OCI DS, IBM DS).

Main risk: breadth of keywords > depth of proof. MLOps words are present (Docker, FastAPI, ONNX, MinIO, pipelines) but there is no experiment tracking, no CI/CD, no tests, no monitoring, no cloud deployment, no data versioning. An interviewer will probe exactly there.

🇫🇷 **Lecture d'un recruteur en 20 secondes :** une étudiante en Réseaux & Télécoms qui a basculé vers le ML, avec un vrai fil conducteur : **données IoT → prévision de séries temporelles → déploiement de modèle optimisé** (AirSentinel). C'est votre identité professionnelle ; le reste est un support.

Forces : réflexe de déploiement (ONNX + FastAPI + Docker), comparaison à un modèle de référence (persistance), résultats chiffrés, profil IoT/réseaux, certifications.

Risque principal : plus de mots-clés que de preuves. Il manque suivi d'expériences, CI/CD, tests, monitoring, cloud, versioning des données.

Reconstructed architecture of AirSentinel (what an interviewer will ask you to draw)

```
IoT sensors (PM2.5, PM10, NO2, temp, humidity...)
      ↓  (HTTP/MQTT/serial -> WebSocket)
Async ingestion (FastAPI WebSocket endpoints, asyncio)
      ↓
Storage / buffer (MinIO object storage? DB? -> YOU MUST be able to say what)
      ↓
Preprocessing (cleaning, resampling, lag/window features, scaling)
      ↓
Transformer model (trained in TensorFlow/Keras or PyTorch)
      ↓  export
ONNX file  ->  ONNX Runtime (inference session)
      ↓
FastAPI prediction service (12 h forecast, <1 s)  [Docker container]
      ↓
Streamlit dashboard + PyDeck 3D maps (role-based auth, offline mode)
```

🇬🇧 You must be able to say, for each arrow: what format the data has, what can fail, and how you tested it. 🇫🇷 Pour chaque flèche : format des données, ce qui peut échouer, comment vous l'avez testé.

---

2. Technical Skill Map

Legend — Priority: 🟢 MUST KNOW · 🟡 SHOULD KNOW · 🔵 NICE TO KNOW · 🔴 NOT NOW

Category	Skill / Technology	Importance for your profile	Priority	What you must know
Language	Python	Very high	🟢	Data structures, functions, OOP basics, venv, async/await, typing, exceptions
Language	SQL	High	🟢	SELECT, WHERE, JOIN, GROUP BY, window functions, indexes, CTE
Language	Java, PHP	Low	🔵	Only to say what you built; do not lead with them

Category	Skill / Technology	Importance for your profile	Priority	What you must know
Data	Pandas, NumPy	Very high	🟢	DataFrame ops, groupby, merge, resample, vectorization, broadcasting
Data	EDA, cleaning, outliers, imputation	High	🟢	Missing values (MCAR/MAR/MNAR), IQR/z-score, imputation leakage
Viz	Matplotlib, Seaborn	Medium	🟡	Choosing the right plot, diagnostic plots (residuals, ACF)
ML	Supervised learning	Very high	🟢	Train/val/test, loss, bias-variance, CV, metrics
ML	Random Forest	High	🟢	Bagging, bootstrap, feature importance, hyperparameters
ML	Feature engineering (lags)	Very high	🟢	Lag/rolling features, no leakage, encoding, scaling
ML	Time series: ARIMA, seasonality	Very high	🟢	Stationarity, differencing, ACF/PACF, p-d-q, decomposition
ML	Scikit-learn	Very high	🟢	Pipeline, ColumnTransformer, fit/transform, GridSearchCV, TimeSeriesSplit
DL	MLP, LSTM	High	🟢	Layers, activations, backprop, vanishing gradient, gates, windows
DL	Transformer	High (AirSentinel)	🟢	Self-attention, positional encoding, encoder, why vs LSTM

Category	Skill / Technology	Importance for your profile	Priority	What you must know
DL	TensorFlow/Keras	High	🟡	Sequential/Functional API, compile/fit, callbacks, save/export
Serving	ONNX / ONNX Runtime	Very high (differentiator)	🟢	Export, opset, graph, execution providers, speed vs framework
Serving	FastAPI	Very high	🟢	Endpoints, Pydantic, async, status codes, OpenAPI, WebSocket
Serving	WebSockets	High	🟡	Handshake upgrade, full-duplex, vs HTTP polling/SSE
DevOps	Docker	Very high	🟢	Image vs container, Dockerfile, volumes, networks, Compose
DevOps	Git/GitHub	High	🟢	branch, merge, rebase, PR, conflict resolution, .gitignore
Storage	MinIO	Medium (claimed)	🟡	S3-compatible object storage, buckets, objects — be ready to justify
App	Streamlit, PyDeck	Medium	🟡	Rerun model, session_state, caching, deck.gl layers
Web	Laravel, Eloquent, Blade, Tailwind, Alpine	Low for ML roles	🔵	MVC, ORM, migrations, auth middleware
DB	MySQL	Medium	🟡	Relational schema, keys, normalization, indexes
Missing	MLflow, DVC	High value	🟢 (learn)	Tracking, registry, data versioning
Missing	CI/CD (GitHub Actions)	High value	🟢 (learn)	Workflow, jobs, tests, build/push image

Category	Skill / Technology	Importance for your profile	Priority	What you must know
Missing	Linux/Bash	High value	🟢 (learn)	Navigation, permissions, processes, ssh, cron
Missing	Testing (pytest)	High value	🟢 (learn)	Unit tests, fixtures, API tests
Missing	Monitoring, drift	High value	🟡 (learn)	Latency, errors, data/model drift
Missing	Kubernetes, Airflow, Spark, Kafka	Medium	🔵	Concepts only for now
Cloud	Oracle OCI (certificate)	Medium	🟡	Be ready to explain what you actually did, not the badge

---

3. Skills I Must Master (honest triage)

🇬🇧 🟢 **MUST (defend in depth):** Python, Pandas/NumPy, scikit-learn, time-series forecasting (ARIMA, lags, leakage, time-based split), regression metrics (RMSE, MAE, R^2, MAPE), Random Forest, MLP/LSTM basics, Transformer intuition, ONNX Runtime, FastAPI, Docker, Git, SQL.

🟡 **SHOULD (discuss confidently):** WebSockets/async, MinIO, Streamlit, TensorFlow/Keras details, MySQL, Oracle OCI content.

🔵 **NICE:** Laravel stack, Java, PHP, PyDeck internals.

🔴 **NOT NOW:** Spark, Kafka, Kubernetes internals, feature stores, Triton/KServe, Terraform. Learn the *vocabulary*, not the tooling.

🇫🇷 🟢 **INDISPENSABLE :** Python, Pandas/NumPy, scikit-learn, séries temporelles (ARIMA, lags, fuite de données, découpage temporel), métriques de régression, Random Forest, MLP/LSTM, intuition Transformer, ONNX Runtime, FastAPI, Docker, Git, SQL.

🟡 **À SAVOIR DISCUTER :** WebSockets/async, MinIO, Streamlit, Keras, MySQL, contenu de la certification OCI. 🔵 **UTILE :** Laravel, Java, PHP. 🔴 **PAS MAINTENANT :** Spark, Kafka, Kubernetes avancé, feature stores.

> Honest rule: ten technologies you can defend beat thirty you cannot. Your CV lists ~35 names. Be ready to say, for each one: 'I used it for X, in project Y, and the hardest problem was Z.'

4. Programming & Software Engineering

4.1 Python (for ML engineering)

🇬🇧 **Level 1 — Interview simple:** Python is the glue language of data/ML. I use it for data cleaning (Pandas), modelling (scikit-learn, Keras), and serving (FastAPI). I write modular code with virtual environments so projects are reproducible.

Level 2 — Technical: Python is interpreted, dynamically typed, with reference-counted memory plus a garbage collector. The GIL means threads do not run Python bytecode in parallel, so CPU-bound work uses multiprocessing or native libraries (NumPy, ONNX Runtime release the GIL), while I/O-bound work uses `asyncio` or threads.

Must-know sub-concepts: list/dict/set complexity · comprehensions · generators (`yield`) · decorators · context managers (`with`) · classes/dataclasses · type hints · exceptions · virtual environments (`venv`, `pip`, `requirements.txt`, `pyproject.toml`) · `if __name__ == '__main__'` · logging (not `print`).

🇫🇷 **Niveau 1 :** Python est le langage de liaison de la data/ML : nettoyage (Pandas), modélisation (scikit-learn, Keras) et déploiement (FastAPI). J'utilise des environnements virtuels pour la reproductibilité.

Niveau 2 : le GIL empêche l'exécution parallèle de threads Python ; pour le calcul intensif on utilise multiprocessing ou des bibliothèques natives (NumPy, ONNX Runtime), et pour les entrées/sorties `asyncio`.

Analogy: Python = a project manager who delegates heavy lifting to specialist workers (C/C++ libraries).

Interview Q&A

- *Beginner — list vs tuple?* → Lists are mutable, tuples immutable and hashable if their contents are. Follow-up: tuples can be dict keys; use them for fixed records.
- *Intermediate — what is a generator and why use it in data pipelines?* → A function that yields values lazily, so you stream large files without loading everything into RAM.
- *Practical — your script works on your laptop but fails on a colleague's.* → Pin dependencies (`requirements.txt`/lockfile), use a venv or Docker, check Python version, avoid absolute paths.
- *Trap — what does the GIL change for your FastAPI ML service?* → Not much for the I/O layer (async), and ONNX Runtime does heavy compute in C++ outside the GIL; but pure-Python preprocessing in the request path can bottleneck, so scale with several worker processes (`uvicorn --workers`).

Essential commands

```
python -m venv .venv && source .venv/bin/activate   # create/activate env
pip install -r requirements.txt                     # install deps
pip freeze > requirements.txt                       # pin versions
python -m pytest                                    # run tests
```

📺 **Videos** | Python for beginners/intermediate | freeCodeCamp 'Python for Everybody' or Corey Schafer Python tutorials | Level: beginner→intermediate | Search: https://www.youtube.com/results?search_query=Corey+Schafer+Python+virtual+environments · https://www.youtube.com/results?search_query=freeCodeCamp+Python+full+course

4.2 async / await (you claim 'pipelines asynchrones')

🇬🇧 **Level 1:** Async lets one process handle many waiting tasks (network, sensors) without blocking. While one request waits for data, the event loop serves another.

Level 2: `async def` creates a coroutine; `await` yields control to the **event loop** (`asyncio`) until the I/O completes. It is cooperative multitasking on a single thread: great for I/O-bound ingestion, useless for CPU-bound inference unless offloaded (`run_in_executor`, a thread pool, or a separate worker).

🇫🇷 **Niveau 1 :** l'asynchrone permet de gérer beaucoup de tâches en attente (réseau, capteurs) sans bloquer. **Niveau 2 :** `await` rend la main à la boucle d'événements ; c'est du multitâche coopératif mono-thread, adapté aux E/S, pas au calcul CPU.

Analogy: a waiter taking many orders and not standing at one table until the dish is ready.

Trap question — 'You use async. What happens if you call a blocking ONNX inference inside an `async def` endpoint?' → It blocks the event loop and freezes every other WebSocket/request. Fix: declare the endpoint as a normal `def` (FastAPI runs it in a thread pool), or `await asyncio.to_thread(session.run, ...)`.

📺 Search: https://www.youtube.com/results?search_query=Python+asyncio+tutorial+Corey+Schafer · https://www.youtube.com/results?search_query=FastAPI+async+await+explained

4.3 Git / GitHub

🇬🇧 **Level 1:** Git tracks code history so I can experiment on branches, collaborate through pull requests, and roll back mistakes. **Level 2:** Git stores snapshots (commits) as a DAG of objects identified by hashes; a branch is just a movable pointer.

🇫🇷 Git garde l'historique du code : branches pour expérimenter, pull requests pour collaborer, retour arrière possible. Une branche est un simple pointeur vers un commit.

Must-know: working tree / staging area / repository · commit · branch · merge vs rebase · pull request · conflict · `.gitignore` (never commit data, `.env`, secrets, models) · tags · Git LFS / DVC for big files.

```
git status / git add -p / git commit -m 'msg'
git switch -c feature/x ; git merge feature/x
git log --oneline --graph ; git diff
git stash ; git restore <file> ; git revert <sha>
```

Trap: 'You pushed a password to GitHub. What now?' → Rotate the secret immediately (deleting the commit is not enough, history is cloned), then purge history (`git filter-repo`), add to `.gitignore`, use env vars / secret managers.

📺 Search: https://www.youtube.com/results?search_query=freeCodeCamp+Git+and+GitHub+for+beginners · https://www.youtube.com/results?search_query=Git+merge+vs+rebase+explained

4.4 Linux & Bash (missing on your CV — fix it)

Containers, servers and CI all run Linux. Know: `ls cd pwd cat less grep find chmod chown ps top kill ssh scp curl tail -f`, pipes `|`, redirection `>`, environment variables, file permissions, `cron`, `systemd` basics. 📺 https://www.youtube.com/results?search_query=freeCodeCamp+Linux+command+line+full+course

---

5. Data Science

5.1 EDA, cleaning, outliers, imputation (California housing, solar data)

🇬🇧 **Level 1:** Before modelling I look at the data: distributions, missing values, outliers, correlations. I clean and impute so the model learns signal, not errors.

Level 2: Missing data types: **MCAR** (random), **MAR** (depends on observed variables), **MNAR** (depends on the missing value itself). Imputation: mean/median/mode, KNN, model-based, forward-fill for time series. **Outliers:** IQR rule (below Q1−1.5·IQR or above Q3+1.5·IQR), z-score, domain rules (a negative PM2.5 reading is a sensor fault, not an outlier). **Golden rule:** fit imputers/scalers on the **training set only**, then apply to validation/test — otherwise **data leakage**.

🇫🇷 **Niveau 1 :** j'explore les données (distributions, valeurs manquantes, aberrations, corrélations) puis je nettoie. **Niveau 2 :** MCAR/MAR/MNAR ; règle IQR, z-score, règles métier ; l'imputation et la normalisation s'ajustent sur le train uniquement pour éviter la fuite de données.

Analogy: cleaning a lab sample before the measurement.

Q&A

- *Intermediate — how did you treat outliers in California housing?* → Be precise: which variable (e.g. capped `median_house_value` at 500k, `total_rooms` skew), which method (IQR/capping/log transform), and why removal vs capping. If you only removed rows, say so and mention the trade-off.
- *Trap — you imputed before train/test split. Problem?* → Yes, leakage: test statistics influenced the training data. Use a `Pipeline` so imputation is fitted inside each CV fold.
- *Trap — is a tree model sensitive to outliers/scaling?* → Random Forest is robust to scaling and fairly robust to outliers in features; linear models, MLP and LSTM are not.

Pandas essentials

```
df.info(); df.describe(); df.isna().mean()
df['date'] = pd.to_datetime(df['date']); df = df.set_index('date')
df.resample('1h').mean()            # regular time grid
df['pm25_lag1'] = df['pm25'].shift(1)
df['pm25_roll24'] = df['pm25'].rolling(24).mean()
df.groupby('station')['pm25'].agg(['mean','std'])
```

📺 StatQuest 'Machine Learning / missing data' + Corey Schafer Pandas series. Search: https://www.youtube.com/results?search_query=Corey+Schafer+Pandas+tutorial · https://www.youtube.com/results?search_query=freeCodeCamp+NumPy+Pandas+data+analysis

5.2 Time-series forecasting (your strongest topic)

🇬🇧 **Level 1:** A time series is data ordered in time. Forecasting predicts future values from past ones. Time order matters, so I never shuffle data randomly and I always compare my model with a naive baseline.

Level 2 — vocabulary you must master:

Term	Meaning
Trend	Long-term direction
Seasonality	Repeating pattern with fixed period (daily solar cycle, 24 h)
Stationarity	Mean/variance/autocorrelation do not change over time; required by ARIMA (check with ADF test)

Term	Meaning
Differencing (d)	Subtract previous value to remove trend
ACF / PACF	Correlation with lags; guide choice of q and p
Lag features	Past values used as inputs (t−1, t−24)
Rolling stats	Moving mean/std over a window
Persistence model	Forecast = last observed value (or same hour yesterday: seasonal naive)
Walk-forward / TimeSeriesSplit	Validation that always trains on the past, tests on the future
Data leakage	Using future information to predict the past

ARIMA(p,d,q): AutoRegressive (p past values) + Integrated (d differencing) + Moving Average (q past errors). **SARIMA** adds seasonal terms. ARIMA is univariate and linear; it struggles with multi-sensor, nonlinear data — that is exactly why you moved to ML/DL in later projects.

🇫🇷 **Niveau 1 :** une série temporelle est une suite de mesures ordonnées dans le temps ; on ne mélange jamais l'ordre et on compare toujours à un modèle naïf. **Niveau 2 :** ARIMA(p,d,q) = autorégression + différenciation + moyenne mobile ; il exige la stationnarité (test ADF) ; SARIMA gère la saisonnalité ; validation par `TimeSeriesSplit`.

Analogy: weather forecasting from yesterday's patterns — and the baseline is saying 'tomorrow will be like today'.

Your numbers (be ready to defend them): ARIMA RMSE 113.54 vs persistence 147.99 → (147.99−113.54)/147.99 ≈ **23.3 % improvement**. ✔ correct arithmetic.

Q&A

- *Beginner — why not use* `train_test_split(shuffle=True)` *?* → It leaks the future into training and breaks temporal dependence.
- *Intermediate — how did you choose p, d, q?* → ADF test → d; ACF/PACF plots → q, p; compare AIC/BIC; or `auto_arima`. Never say 'I tried random values'.
- *Practical — solar production is 0 at night; RMSE looks fine but is it?* → Night zeros inflate performance. Evaluate daytime only, or compare to seasonal-naive (same hour yesterday), which is a **much tougher baseline** than last-value persistence. Expect this question: is 23 % over persistence impressive when the series has a strong 24 h cycle?
- *Trap — is RMSE of 113.54 good?* → Meaningless without scale. Give the unit (kW? MW?), the mean/range of the target, and compare with MAE, MAPE/sMAPE or normalized RMSE.

- *Trap — $R^2 = 0.85$ on housing: is that good?* → Depends on baseline, irreducible noise and cross-validation; ask whether it is test-set R^2 and whether the target was capped.

📺 StatQuest + Krish Naik time series. Search: https://www.youtube.com/results?search_query=ARIMA+time+series+forecasting+explained+python · https://www.youtube.com/results?search_query=time+series+stationarity+ACF+PACF+explained · https://www.youtube.com/results?search_query=Krish+Naik+time+series+ARIMA

5.3 Evaluation metrics (regression)

Metric	Formula idea	Use / trap
MAE	mean(abs error)	Same unit as target, robust, easy to explain
MSE / RMSE	mean(squared error), then sqrt	Penalizes large errors; sensitive to outliers
R^2	1 − SSres/SStot	Share of variance explained; can mislead on time series
MAPE	mean(abs error / abs y)	Explodes when y ≈ 0 (night solar, clean-air readings) — use sMAPE/MAE

🇫🇷 RMSE pénalise fortement les grosses erreurs ; MAE est plus robuste ; MAPE est dangereux quand la cible approche zéro ; R^2 mesure la variance expliquée mais peut tromper en séries temporelles.

📺 StatQuest: https://www.youtube.com/results?search_query=StatQuest+R-squared · https://www.youtube.com/results?search_query=regression+metrics+MAE+MSE+RMSE+explained

---

6. Machine Learning

6.1 Supervised learning workflow

🇬🇧 **Level 1:** I give the model examples with known answers (features X, target y). It learns a mapping, and I measure how well it generalizes to data it never saw.

Level 2: Dataset → features/target → split (train/validation/test) → preprocessing → model → loss function → optimization → evaluation → cross-validation → hyperparameter tuning → final test → deployment. **Overfitting** = great on train, poor on new data (high variance). **Underfitting** = poor everywhere (high bias). **Bias–variance trade-off** is the vocabulary interviewers love.

🇫🇷 **Niveau 1 :** je donne au modèle des exemples avec la réponse connue ; il apprend la relation et je mesure sa capacité à généraliser. **Niveau 2 :** surapprentissage = excellent en train, mauvais ailleurs (variance élevée) ; sous-apprentissage = mauvais partout (biais élevé).

Analogy: studying past exams (train), checking with mock exams (validation), then the real exam (test) — you never peek at the real exam.

Q&A

- *Beginner — train vs validation vs test?* → Train fits parameters, validation tunes hyperparameters/choose model, test gives one final unbiased estimate.
- *Intermediate — how do you detect overfitting?* → Gap between train and validation error; learning curves; fix with regularization, more data, simpler model, early stopping, dropout.
- *Practical — test score is much worse than CV score.* → Check leakage in CV, distribution shift, time-order violations, tiny test set.
- *Trap — 'R² = 0.85 so the model is excellent.'* → Not alone: check baseline (e.g. predicting the mean or a linear model), residual patterns, errors by segment (coastal vs inland), and that median income is a very strong, almost-trivial predictor.

scikit-learn essentials

```
from sklearn.model_selection import train_test_split, GridSearchCV,
TimeSeriesSplit
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestRegressor

pre = ColumnTransformer([
  ('num', Pipeline([('imp', SimpleImputer(strategy='median')),('sc',
StandardScaler())]), num_cols),
  ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)])
model = Pipeline([('pre', pre), ('rf',
RandomForestRegressor(n_estimators=300, random_state=42))])
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

🇬🇧 `fit` learns parameters from data; `transform` applies them; `Pipeline` guarantees the same steps in training and production and prevents leakage. 🇫🇷 `fit` apprend, `transform` applique, `Pipeline` garantit les mêmes étapes en entraînement et en production.

📺 StatQuest 'Machine Learning Fundamentals' playlist; freeCodeCamp scikit-learn; Krish Naik ML. Search: https://www.youtube.com/results?search_query=StatQuest+bias+and+variance · https://www.youtube.com/results?search_query=StatQuest+cross+validation · https://www.youtube.com/results?search_query=freeCodeCamp+scikit-learn+crash+course

6.2 Random Forest

🇬🇧 **Level 1:** Many decision trees, each trained on a random sample of rows and features; their predictions are averaged (regression) or voted (classification). Averaging reduces variance, so it overfits less than a single tree.

Level 2: Bagging (bootstrap aggregating) + random feature subset at each split (`max_features`) decorrelates trees. Key hyperparameters: `n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`. **OOB score** gives a free validation estimate. **Feature importance:** impurity-based (biased toward high-cardinality/continuous features) vs permutation importance (more reliable). Trees cannot extrapolate beyond the training range — important for time series with trend.

🇫🇷 **Niveau 1 :** une forêt de nombreux arbres entraînés sur des échantillons aléatoires ; on moyenne leurs prédictions, ce qui réduit la variance. **Niveau 2 :** bagging + sous-ensemble aléatoire de variables ; score OOB ; importance par permutation plus fiable que l'importance par impureté ; un arbre ne sait pas extrapoler hors de la plage d'entraînement.

Analogy: asking 300 slightly different experts and taking the average opinion.

Q&A

- *Intermediate — why did RF win for humidity in your IoT benchmark?* → Humidity depends on nonlinear interactions with temperature and recent lags; RF with lag features captures this with little tuning and small data, while LSTMs need more data and tuning.
- *Trap — 'median income and ocean proximity are the main factors' — causal?* → No: feature importance shows predictive contribution, not causation; correlated features share importance.
- *Trap — RF vs Gradient Boosting (XGBoost/LightGBM)?* → Boosting trains trees sequentially to correct errors, usually more accurate on tabular data but more sensitive to tuning. Know that this is the modern default on tabular problems and **add one XGBoost/LightGBM model to your portfolio**.

📺 StatQuest: https://www.youtube.com/results?search_query=StatQuest+Random+Forests+Part+1 · https://www.youtube.com/results?search_query=StatQuest+XGBoost

6.3 Feature engineering & model benchmarking

🇬🇧 Feature engineering turns raw data into inputs a model can exploit: lags, rolling means, hour-of-day/day-of-week (cyclical encoding with sin/cos), differences, interaction terms, encodings. **Benchmarking** = compare models fairly: same split, same metrics, same preprocessing, multiple seeds.

🇫🇷 Le feature engineering transforme les données brutes en variables exploitables : lags, moyennes glissantes, heure/jour (encodage cyclique sin/cos), différences. Un benchmark compare les modèles équitablement : même découpage, mêmes métriques, plusieurs graines.

Questions on your IoT benchmark (very likely):

- *Why 'RMSE 0.695–1.109' as a range?* → Be ready: range across horizons, sensors or folds? Say it explicitly.
- *How many seeds / folds? Any statistical significance?* → If single run, admit it and say you would repeat with several seeds and use time-series CV.
- *Was the MLP given lag features like the RF?* → Fair comparison requires the same information. If the MLP got lags and the LSTM got raw windows, explain.
- *Trap — 'MLP best for temperature, RF for humidity': why not one model?* → Different physical dynamics: temperature is smooth and strongly periodic; humidity is noisier and nonlinear. Show you understand the signal, not only the scores.

---

7. Deep Learning / NLP / Computer Vision (only what your CV contains)

Your CV has Deep Learning (MLP, LSTM, Transformer). No NLP or CV project appears, so I skip them — do not claim them.

7.1 Neural network basics (MLP)

🇬🇧 **Level 1:** An MLP stacks layers of neurons; each computes a weighted sum plus a nonlinearity. Training adjusts weights to minimize the loss. **Level 2:** Forward pass → loss (MSE for regression) → **backpropagation** (chain rule gives gradients) → optimizer (SGD/Adam) updates weights. Activations: ReLU, tanh; without them the network is just linear. Key knobs: learning rate, batch size, epochs, layers/units, dropout, early stopping, normalization.

🇫🇷 Un MLP empile des couches de neurones (somme pondérée + non-linéarité). L'entraînement minimise la perte via la rétropropagation et un optimiseur (Adam, SGD). Sans activation, le réseau reste linéaire.

Analogy: a team of juniors each giving a weighted opinion, then a manager combining them, corrected by feedback after every mistake.

📺 3Blue1Brown: https://www.youtube.com/results?search_query=3Blue1Brown+neural+networks+chapter+1 · 3Blue1Brown backpropagation: https://www.youtube.com/results?search_query=3Blue1Brown+backpropagation

7.2 LSTM

🇬🇧 **Level 1:** An LSTM is a recurrent network that reads a sequence step by step and keeps a memory, so it can use information from many steps back. **Level 2:** A cell state carries information; **forget, input and output gates** decide what to erase, write and expose. This mitigates the **vanishing gradient** of plain RNNs. Input shape: `(batch, timesteps, features)`. You must build sliding windows from your series and scale data (fit scaler on train only).

🇫🇷 Un LSTM lit une séquence pas à pas avec une mémoire interne ; les portes d'oubli, d'entrée et de sortie gèrent ce qui est conservé, ce qui atténue le problème du gradient qui disparaît. Forme d'entrée : `(batch, pas de temps, variables)`.

Analogy: reading a book and keeping running notes, crossing out outdated ones.

Q&A

- *Why did LSTM not win in your benchmark?* → Small/medium data, sensitivity to window size and tuning, little long-term dependency beyond what lag features already capture. Honest answer > defending the model.
- *Trap — how do you build the input for 12-hour forecasts?* → Direct (one output per horizon, multi-output), recursive (feed predictions back; errors accumulate), or seq2seq. Know which you used.

📺 Search: https://www.youtube.com/results?search_query=StatQuest+LSTM · https://www.youtube.com/results?search_query=Understanding+LSTM+networks+explained+visually

7.3 Transformer (AirSentinel) — the one they will drill

🇬🇧 **Level 1:** A Transformer processes the whole sequence at once and uses **attention** to decide which time steps matter for each prediction. It trains faster in parallel than an LSTM and captures long-range dependencies. **Level 2:** Input embedding/projection + **positional encoding** (order information) → stacked blocks of **multi-head self-attention** (Q, K, V: attention = softmax($QK^T/\sqrt{d}$)·V) + feed-forward + residual connections + layer

norm. For forecasting you often use encoder-only models with a regression head, or architectures like Informer/Autoformer/PatchTST. Cost of attention is $O(n^2)$ in sequence length.

🇫🇷 Un Transformer traite toute la séquence en parallèle et utilise l'attention pour déterminer quels pas de temps comptent ; l'encodage positionnel conserve l'ordre. L'attention se calcule par $\text{softmax}(QK^T/\sqrt{d})V$; le coût est quadratique en longueur.

Analogy: instead of reading a diary line by line (LSTM), you see the whole page and highlight the lines relevant to today's question.

Q&A

- *Beginner — what is attention?* → A learned weighting of how much each time step contributes to each output.
- *Intermediate — why a Transformer for air-quality forecasting over LSTM?* → Parallel training, long dependencies, multi-sensor context. Be ready to admit that on small tabular time series, simpler models (RF, GBM) often match it; your benchmark project actually supports that nuance — use it.
- *Project — which framework trained it, how many parameters, what window and horizon, what dataset size, what RMSE versus your baselines?* → Have the exact numbers in your head. **If AirSentinel's accuracy is not on your CV, an interviewer will ask for it first.**
- *Trap — 'Transformer optimized with ONNX': what did you optimize?* → Graph optimizations (operator fusion, constant folding), optionally quantization (INT8/FP16), and a C++ runtime without Python framework overhead. Show before/after latency numbers.

📺 Karpathy: https://www.youtube.com/results?search_query=Andrej+Karpathy+Let%27s+build+GPT+from+scratch · 3Blue1Brown attention: https://www.youtube.com/results?search_query=3Blue1Brown+attention+in+transformers · StatQuest: https://www.youtube.com/results?search_query=StatQuest+Transformers

7.4 TensorFlow / Keras

```
model = keras.Sequential([layers.Input((24, n_feat)), layers.LSTM(64),
layers.Dense(12)])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.fit(X_tr, y_tr, validation_data=(X_val, y_val), epochs=50,
          callbacks=[keras.callbacks.EarlyStopping(patience=5,
restore_best_weights=True)])
model.save('model.keras')
```

🇬🇧 `compile` sets optimizer/loss; `fit` trains; callbacks like EarlyStopping avoid overfitting. Alternative note: PyTorch dominates research and many industry jobs — learn the basics of it, because many interviewers use PyTorch vocabulary. 🇫🇷 `compile` définit optimiseur et perte, `fit` entraîne ; EarlyStopping limite le surapprentissage. PyTorch domine la recherche et beaucoup d'offres : apprenez-en les bases.

📺 freeCodeCamp: https://www.youtube.com/results?search_query=freeCodeCamp+TensorFlow+Keras+course · https://www.youtube.com/results?search_query=freeCodeCamp+PyTorch+full+course

---

8. Data Engineering

8.1 Data pipelines, ingestion & streaming

🇬🇧 **Level 1:** A data pipeline moves data from a source to a destination while cleaning and transforming it so it is ready for analysis or models. **Level 2:** Source → ingestion (batch or streaming) → validation → transformation → storage → serving. **Batch** processes chunks periodically; **streaming** processes events continuously. Core ideas: schema, idempotency (rerun gives the same result), late/out-of-order data, backpressure, retries, dead-letter queue, data quality checks.

🇫🇷 Un pipeline de données déplace et transforme les données de la source vers la destination. Batch = par lots périodiques ; streaming = en continu. Notions clés : schéma, idempotence, données tardives, réessais, contrôles qualité.

Analogy: a water network: sources, pipes, filters, tanks, taps.

Q&A (about AirSentinel)

- *What happens when a sensor sends malformed or late data, or disconnects?* → Validate with Pydantic schema, discard/quarantine bad messages, handle reconnects, timestamp from device vs server, fill gaps (resample + interpolation with a max gap).
- *Why WebSockets instead of HTTP POST per reading?* → One persistent full-duplex connection, lower overhead and latency for high-frequency streams; but harder to scale/load-balance. For IoT at scale, MQTT or Kafka are the industry standard — know why you did not need them.
- *Trap — 'your pipeline is real-time': define latency.* → End-to-end from sensor to dashboard (ingest + inference + render), measured at p50/p95.

📺 DataTalksClub Data Engineering Zoomcamp: https://www.youtube.com/results?search_query=DataTalksClub+Data+Engineering+Zoomcamp · https://www.youtube.com/results?search_query=data+pipeline+explained+batch+vs+streaming

8.2 MinIO / object storage

🇬🇧 **Level 1:** MinIO is self-hosted object storage that speaks the Amazon S3 API: you store files (objects) in buckets instead of folders on a disk. **Level 2:** Objects = data + metadata + key; flat namespace; immutable writes; accessed via HTTP/S3 SDK (`boto3` , `minio` client) with access/secret keys; ideal for datasets, model artifacts, logs; scales horizontally; local equivalent of AWS S3.

🇫🇷 MinIO est un stockage objet auto-hébergé compatible S3 : on stocke des objets dans des buckets. Idéal pour datasets, artefacts de modèles et logs ; équivalent local d'AWS S3.

Analogy: a giant warehouse with labeled boxes, retrieved by label, not by shelf location.

> ⚠️ **Credibility flag:** MinIO appears in your Skills but in no experience bullet. A recruiter will ask 'what did you store in MinIO?'. Either add it to the AirSentinel bullet (e.g. raw sensor history, ONNX model versions) or remove it. Same for 'pipelines de données': be ready to describe one concretely.

```
from minio import Minio
c = Minio('minio:9000', access_key='...', secret_key='...', secure=False)
c.fput_object('models', 'airsentinel/v1/model.onnx', 'model.onnx')
```

📺 Search: https://www.youtube.com/results?search_query=MinIO+tutorial+S3+compatible+object+storage · https://www.youtube.com/results?search_query=object+storage+vs+file+storage+vs+block+storage

---

9. APIs & Backend

9.1 HTTP, REST & FastAPI

🇬🇧 **Level 1:** An API lets one program ask another to do something. FastAPI is a Python framework to build fast, typed REST APIs; I use it to expose my ML model as a service others can call. **Level 2:** HTTP request = method + URL + headers + body; response = status code + headers + body. REST uses resources and verbs (GET read, POST create/compute, PUT/PATCH update, DELETE remove). FastAPI uses **Starlette** (ASGI) + **Pydantic** (validation/serialization) and auto-generates **OpenAPI/Swagger** docs at `/docs` . Run with **Uvicorn** (ASGI server).

🇫🇷 **Niveau 1 :** une API permet à un programme d'en solliciter un autre ; FastAPI sert à exposer un modèle ML sous forme de service. **Niveau 2 :** requête HTTP = méthode + URL + en-têtes + corps ; réponse = code statut + corps ; FastAPI s'appuie sur Starlette (ASGI)

et Pydantic (validation), et génère la doc OpenAPI automatiquement.

Analogy: a restaurant: the menu (OpenAPI), the waiter (HTTP), the kitchen (model), the order form checked before it goes to the kitchen (Pydantic).

Status codes: 200 OK · 201 Created · 400/422 invalid input · 401 unauthenticated · 403 forbidden · 404 not found · 429 too many requests · 500 server error · 503 unavailable.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import numpy as np, onnxruntime as ort

app = FastAPI(title='AirSentinel')
sess = ort.InferenceSession('model.onnx', providers=['CPUExecutionProvider'])
# load ONCE at startup

class Window(BaseModel):
    values: list[list[float]] = Field(..., description='(24, n_features)')

@app.get('/health')
def health(): return {'status': 'ok'}

@app.post('/predict')
def predict(w: Window):
    x = np.asarray([w.values], dtype=np.float32)
    if x.shape[1:] != (24, 8): raise HTTPException(422, 'bad shape')
    out = sess.run(None, {sess.get_inputs()[0].name: x})[0]
    return {'forecast_12h': out[0].tolist()}
```

```
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 2
```

Q&A

- *Beginner — GET vs POST?* → GET retrieves and is idempotent/cacheable; POST sends data and triggers processing; prediction is POST because inputs are in the body.
- *Intermediate — why load the model at startup and not inside the endpoint?* → Loading costs hundreds of ms–seconds; do it once (lifespan event) and reuse the session.
- *Practical — the API gets slow under load.* → Measure first (p95 latency); check blocking calls in async endpoints, add workers, batch requests, cache, limit input size, scale replicas behind a load balancer.
- *Trap — 'sub-second forecasts': on what hardware, with what batch size, measured how?* → Give CPU/GPU, warm-up excluded, p50/p95 over N requests, and include/exclude network.

- *Trap — authentication?* → Your dashboard has role-based access: explain JWT/session, hashing (bcrypt/argon2), roles vs permissions, and that Streamlit auth alone is not API security.

📺 Search: https://www.youtube.com/results?search_query=freeCodeCamp+FastAPI+course · https://www.youtube.com/results?search_query=Bug+Bytes+OR+ArjanCodes+FastAPI+machine+learning+deployment · https://www.youtube.com/results?search_query=REST+API+explained+HTTP+methods+status+codes

9.2 WebSockets

🇬🇧 **Level 1:** A WebSocket is a long-lived two-way connection between client and server, so data can be pushed instantly without repeated requests. **Level 2:** Starts as an HTTP request with `Upgrade: websocket`; server replies 101 Switching Protocols; then full-duplex frames flow over one TCP connection. Alternatives: HTTP polling (wasteful), Server-Sent Events (one-way), MQTT (pub/sub for IoT).

🇫🇷 Un WebSocket est une connexion bidirectionnelle persistante ; la communication démarre en HTTP avec `Upgrade: websocket`, puis les trames circulent dans les deux sens. Alternatives : polling, SSE, MQTT.

```
@app.websocket('/ws/ingest')
async def ingest(ws: WebSocket):
    await ws.accept()
    try:
        while True:
            msg = await ws.receive_json()      # sensor reading
            await queue.put(msg)               # decouple ingestion from
processing
    except WebSocketDisconnect:
        pass
```

Analogy: a phone call (WebSocket) vs sending letters (HTTP).

Trap: 'What happens with 500 concurrent sensors?' → Many open connections, memory/FD limits, heartbeat/ping, reconnect logic, sticky sessions behind a load balancer, backpressure through a queue.

📺 Search: https://www.youtube.com/results?search_query=WebSockets+in+100+seconds+Fireship · https://www.youtube.com/results?search_query=FastAPI+WebSocket+tutorial

9.3 ONNX & ONNX Runtime

🇬🇧 **Level 1:** ONNX is an open, framework-independent file format for trained models. ONNX Runtime is a fast engine that runs those files, so I can train in TensorFlow/PyTorch and serve without the heavy training framework. **Level 2:** Model = computation graph of standard operators (opset version). Export with `tf2onnx` / `torch.onnx.export`. ORT applies graph optimizations (fusion, constant folding), picks **execution providers** (CPU, CUDA, TensorRT, OpenVINO), supports quantization and dynamic axes (variable batch size).

🇫🇷 ONNX est un format ouvert de modèle indépendant du framework ; ONNX Runtime l'exécute rapidement. Graphe d'opérateurs (opset), providers d'exécution (CPU, CUDA…), quantification, axes dynamiques.

Analogy: exporting a Word file to PDF: anyone can read it without Word.

```
# TF -> ONNX
python -m tf2onnx.convert --saved-model saved_model --output model.onnx --
opset 17
# inference
sess = ort.InferenceSession('model.onnx')
out = sess.run(None, {'input': x.astype(np.float32)})
```

Q&A: *Why ONNX rather than serving the Keras model directly?* → Smaller dependency footprint, lower latency, portability, hardware flexibility. *How did you validate the exported model?* → Compare ONNX outputs vs original on the same inputs (`np.allclose`, tolerance 1e-4/1e-5). *Trap — what breaks in conversion?* → Unsupported ops, dynamic shapes, opset mismatch, dtype float64 vs float32. *Alternatives:* TorchScript, TensorRT, OpenVINO, TFLite, Triton, BentoML.

📺 Search: https://www.youtube.com/results?search_query=ONNX+Runtime+tutorial+convert+model+inference · https://www.youtube.com/results?search_query=what+is+ONNX+explained

9.4 Streamlit & PyDeck

🇬🇧 Streamlit builds data apps in pure Python; the script **reruns top-to-bottom on every interaction**, so use `st.session_state` for memory and `@st.cache_data` / `@st.cache_resource` for performance. PyDeck wraps deck.gl to render GPU-accelerated geospatial layers (ColumnLayer/HeatmapLayer for pollution). **Trap:** 'role-based authentication in Streamlit': Streamlit is not a security boundary — enforce access in the API/backend, not only in the UI. 'Offline mode': say exactly what is cached and how it is

refreshed. 🇫🇷 Streamlit relance le script à chaque interaction : `session_state` pour l'état, `cache_data` / `cache_resource` pour la performance. L'authentification doit être appliquée côté backend, pas seulement dans l'interface.

📺 Search: https://www.youtube.com/results?search_query=Streamlit+crash+course+session+state+caching · https://www.youtube.com/results?search_query=PyDeck+3D+map+tutorial

---

10. Docker & Containerization

🇬🇧 **Level 1:** Docker packages an application with its dependencies into a container, so it runs the same on my laptop, a server or the cloud. **Level 2:** Containers are isolated processes sharing the host kernel (Linux namespaces for isolation, cgroups for resource limits). They are lighter than virtual machines, which each run a full guest OS.

🇫🇷 **Niveau 1 :** Docker empaquette une application et ses dépendances dans un conteneur pour qu'elle s'exécute de la même façon partout. **Niveau 2 :** les conteneurs sont des processus isolés qui partagent le noyau de l'hôte (namespaces + cgroups) ; plus légers que des machines virtuelles.

Analogy: a shipping container: whatever is inside, ships, cranes and trucks handle it the same way.

```
Dockerfile -> (docker build) -> Image -> (docker run) -> Container
                                  |                          |
                              Registry (Docker Hub/GHCR)   Network + Volume + Env vars
                                  |
         Compose (many containers) -> CI/CD build+push -> Server / Cloud / Kubernetes
```

Concept	🇬🇧 Meaning	🇫🇷 Signification
Image	Immutable layered template (read-only)	Modèle immuable en couches
Container	Running instance of an image	Instance en cours d'exécution
Dockerfile	Recipe to build an image	Recette de construction
Registry	Image store (Docker Hub, GHCR, OCI Registry)	Dépôt d'images
Port mapping	`-p 8000:8000` host:container	Publication de port

Concept	🇬🇧 Meaning	🇫🇷 Signification
Env variables	Configuration/secrets at runtime (`-e` , `.env`)	Variables d'environnement
Volume	Persistent storage managed by Docker	Stockage persistant géré par Docker
Bind mount	Host folder mounted in container (dev)	Dossier de l'hôte monté
Network	Private virtual network; containers reach each other by service name	Réseau virtuel ; résolution par nom de service
Compose	Defines multi-container apps in `compose.yaml`	Application multi-conteneurs
`depends_on`	Start order only (not readiness)	Ordre de démarrage seulement
Health check	Command proving the app is actually ready	Vérification d'état
`.dockerignore`	Excludes files from build context	Exclut des fichiers du build
Multi-stage build	Build in one stage, copy artifacts to a slim runtime stage	Build en plusieurs étapes, image finale allégée

Dockerfile for your FastAPI + ONNX service

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app/
COPY model/model.onnx ./model/model.onnx
EXPOSE 8000
HEALTHCHECK CMD python -c "import
urllib.request;urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why this order? Docker caches layers: copy `requirements.txt` first so dependency installation is reused when only code changes. `--host 0.0.0.0` is mandatory, otherwise the app is unreachable from outside the container.

compose.yaml (AirSentinel-like)

```
services:
  api:
    build: ./api
    ports: ['8000:8000']
    environment:
      - MINIO_ENDPOINT=minio:9000
    depends_on:
      minio:
        condition: service_healthy
  dashboard:
    build: ./dashboard
    ports: ['8501:8501']
    environment:
      - API_URL=http://api:8000
  minio:
    image: minio/minio
    command: server /data --console-address ':9001'
    volumes:
      - minio_data:/data
    healthcheck:
      test: ['CMD', 'mc', 'ready', 'local']
volumes:
  minio_data:
```

Essential commands

```
docker build -t airsentinel-api:1.0 .      # build image
docker run -d -p 8000:8000 --name api airsentinel-api:1.0
docker ps / docker ps -a / docker images   # list
docker logs -f api                         # follow logs
docker exec -it api bash                   # shell inside container
docker stop api && docker rm api           # stop / remove
docker pull / docker push user/img:tag     # registry
docker compose up -d --build / docker compose down -v
```

Q&A

- *Beginner — image vs container?* → Image = immutable template; container = running instance. Many containers can start from one image.
- *Beginner — container vs VM?* → Containers share the host kernel and start in seconds; VMs include a full OS.
- *Intermediate — why did your container not respond on localhost:8000?* → Wrong port mapping, app bound to 127.0.0.1 instead of 0.0.0.0, container crashed (check `docker logs`).

- *Intermediate — data disappears when the container is removed.* → Writable layer is ephemeral; use volumes.
- *Practical — reduce a 3 GB image.* → Slim/distroless base, multi-stage, `--no-cache-dir`, `.dockerignore`, CPU-only ONNX Runtime instead of full TensorFlow (a real argument for ONNX in your project).
- *Trap — does* `depends_on` *wait until the DB is ready?* → No, only start order, unless `condition: service_healthy` with a health check.
- *Trap — where do you put secrets?* → Not in the image or Git. Use env vars/ `.env` ignored by Git, Docker/Compose secrets, or a cloud secret manager.
- *Trap — does your container run as root?* → By default yes; better to add a non-root `USER`.

Alternatives: Podman (daemonless), containerd, Buildah. **Next step:** Kubernetes (below).

📺 **Videos** | Docker | TechWorld with Nana 'Docker Tutorial for Beginners' / freeCodeCamp 'Docker Tutorial for Beginners' | Beginner | Search: https://www.youtube.com/results?search_query=TechWorld+with+Nana+Docker+tutorial+for+beginners · Compose: https://www.youtube.com/results?search_query=TechWorld+with+Nana+Docker+Compose+tutorial · ML deployment: https://www.youtube.com/results?search_query=deploy+machine+learning+model+FastAPI+Docker

Kubernetes (concept only, 🔵 NICE)

🇬🇧 Kubernetes orchestrates containers across machines: it schedules them, restarts failed ones, scales them and routes traffic. Vocabulary: Cluster → Node → Pod → Deployment → ReplicaSet → Service → Ingress → ConfigMap/Secret → Namespace → probes (liveness/readiness) → requests/limits → HPA autoscaling → `kubectl`. 🇫🇷 Kubernetes orchestre des conteneurs sur plusieurs machines : planification, redémarrage, mise à l'échelle, routage. Apprenez le vocabulaire ; ne prétendez pas le maîtriser sans projet.

```
kubectl get pods / kubectl apply -f deployment.yaml / kubectl logs <pod> /
kubectl describe pod <pod> / kubectl scale deploy api --replicas=3
```

📺 https://www.youtube.com/results?search_query=TechWorld+with+Nana+Kubernetes+tutorial+for+beginners · https://www.youtube.com/results?search_query=KodeKloud+Kubernetes+for+beginners

---

11. MLOps

🇬🇧 **Level 1:** MLOps is DevOps for machine learning: the practices that take a model from a notebook to a reliable, monitored, reproducible production service. **Level 2:** ML systems fail in ways normal software does not: data changes (**data drift**), the relation between inputs and target changes (**concept drift**), and results depend on code + data + parameters + environment. So you version all four.

🇫🇷 **Niveau 1 :** le MLOps est le DevOps du machine learning : amener un modèle du notebook vers un service fiable, surveillé et reproductible. **Niveau 2 :** on versionne code, données, paramètres et environnement, car les données changent (data drift) et la relation entrée-cible aussi (concept drift).

```
Data -> Versioning (DVC) -> Training -> Experiment tracking (MLflow) -> Model
Registry
  -> Packaging (ONNX / Docker) -> CI/CD (GitHub Actions) -> Serving (FastAPI)
  -> Monitoring (latency, errors, drift: Prometheus/Grafana, Evidently) ->
Retraining trigger -> (loop)
```

What your CV honestly covers: Docker, FastAPI, ONNX Runtime = *serving* part. **Missing:** tracking, registry, data versioning, CI/CD, monitoring, retraining. A recruiter reading 'MLOps et déploiement' will expect more than serving. Either add MLflow + GitHub Actions in a project, or rename the heading to 'Déploiement de modèles (ML serving)'.

11.1 MLflow

🇬🇧 MLflow records experiments so results are comparable and reproducible. Vocabulary: **Experiment** (a group) → **Run** (one training execution) → **Parameters** (inputs: lr, depth) → **Metrics** (RMSE per epoch) → **Artifacts** (files: plots, model) → **Model** (flavors: sklearn, onnx, pyfunc) → **Model Registry** (versions + stages/aliases like champion) → **Tracking Server** (UI + backend store + artifact store, e.g. MinIO/S3 — a natural fit with your MinIO claim).

🇫🇷 MLflow enregistre les expériences : expérience → run → paramètres → métriques → artefacts → modèle → registre de modèles (versions, alias) → serveur de tracking. Le stockage d'artefacts peut être MinIO/S3.

Analogy: a lab notebook that records every experiment automatically.

```
import mlflow
mlflow.set_experiment('airsentinel')
with mlflow.start_run():
    mlflow.log_params({'window': 24, 'model': 'transformer'})
    mlflow.log_metric('rmse', rmse)
    mlflow.log_artifact('residuals.png')
    mlflow.sklearn.log_model(model, 'model', registered_model_name='pm25')
```

```
mlflow ui --port 5000
```

Q&A: *Why not just a spreadsheet?* → Automatic, linked to code version and artifacts, queryable, comparable, with registry. *How do you promote a model to production?* → Registry alias/stage after tests and metrics gates, then CI/CD deploys that version. 📺 https://www.youtube.com/results?search_query=MLflow+tutorial+experiment+tracking+model+registry · https://www.youtube.com/results?search_query=DataTalksClub+MLOps+Zoomcamp+MLflow

11.2 DVC (data versioning)

🇬🇧 DVC puts large data/models under version control: Git stores small `.dvc` pointer files, the real data lives in remote storage (S3, MinIO, Drive). **Pipelines** (`dvc.yaml`) define stages (prepare → train → evaluate) with dependencies, so `dvc repro` re-runs only what changed. 🇫🇷 DVC versionne données et modèles volumineux : Git garde des fichiers pointeurs, les données vivent dans un stockage distant ; `dvc repro` relance uniquement les étapes modifiées.

```
dvc init ; dvc add data/raw.csv ; dvc remote add -d storage s3://bucket ; dvc push ; dvc repro
```

📺 https://www.youtube.com/results?search_query=DVC+data+version+control+tutorial

11.3 Monitoring & drift

🇬🇧 Monitor three layers: **system** (latency, CPU/RAM, errors), **data** (input distributions, missing values, schema), **model** (prediction quality once true labels arrive, drift). For air quality the ground truth arrives later (the next hours' sensor readings), so you can compute live forecast error automatically — a strong story for AirSentinel. Tools: Prometheus + Grafana, Evidently AI. Drift tests: PSI, KS test. 🇫🇷 Surveiller trois niveaux : système (latence, erreurs), données (distributions, schéma), modèle (qualité, dérive). Pour AirSentinel, la vérité terrain arrive quelques heures plus tard : on peut donc calculer l'erreur réelle en continu.

Trap: 'How do you know your model is getting worse in production?' → Compare predictions with realized values (rolling RMSE/MAE), alert above a threshold, check input drift, trigger retraining.

📺 https://www.youtube.com/results?search_query=ML+model+monitoring+data+drift+Evidently+tutorial · https://www.youtube.com/results?search_query=Prometheus+Grafana+tutorial+FastAPI

---

12. Cloud

🇬🇧 **Level 1:** Cloud means renting compute, storage and managed services over the internet instead of owning servers. **Level 2:** Models: IaaS (VMs), PaaS (managed platforms), SaaS. Core building blocks on any provider: compute (VM / container service), object storage (S3-like), managed database, networking (VPC, security groups), IAM (identities, roles, least privilege), billing/cost control.

🇫🇷 Le cloud, c'est louer calcul, stockage et services managés par Internet. Briques : calcul, stockage objet, base managée, réseau (VPC), IAM (moindre privilège), gestion des coûts.

Your credential: *Oracle OCI Data Science Professional (April 2026).* Be ready to explain what it covered: OCI Data Science service (notebook sessions, jobs, model catalog, model deployment), Object Storage, IAM policies, networking. Interviewers distrust certificates not backed by a hands-on project. Equivalent services map: S3 ↔ OCI Object Storage ↔ Azure Blob ↔ GCS; SageMaker ↔ OCI Data Science ↔ Azure ML ↔ Vertex AI.

Q&A: *What did you deploy on OCI?* (answer with a real artifact or say it was lab-based). *Difference between a VM and a container service?* → VM = you manage the OS; container service = you manage images, the provider runs the host. *Trap — how do you secure an API key?* → IAM roles/secret manager, never hard-coded.

📺 https://www.youtube.com/results?search_query=cloud+computing+explained+IaaS+PaaS+SaaS · https://www.youtube.com/results?search_query=freeCodeCamp+AWS+Certified+Cloud+Practitioner · https://www.youtube.com/results?search_query=Oracle+OCI+Data+Science+service+tutorial

---

13. DevOps / CI/CD

🇬🇧 **Level 1:** CI automatically tests and builds code on every push; CD automatically delivers it to an environment. **Level 2:** A pipeline = trigger → jobs → steps. For ML: lint → unit tests → data/schema tests → train smoke test → build Docker image → push to

registry → deploy → smoke test the endpoint.

🇫🇷 La CI teste et construit automatiquement le code à chaque push ; la CD le livre vers un environnement. Pour le ML : lint → tests → test de schéma de données → build Docker → push → déploiement → test de fumée.

```
# .github/workflows/ci.yml
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt pytest ruff
      - run: ruff check . && pytest -q
  docker:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t ghcr.io/${{ github.repository }}:${{ github.sha
}} .
```

Q&A: *Difference between continuous delivery and deployment?* → Delivery = always releasable, human approves release; deployment = fully automatic. *What do you test in an ML API?* → `/health`, input validation (422), output shape, a golden-input regression, latency budget. *Trap — can you unit test a model?* → Yes: shapes, no NaN outputs, deterministic with fixed seed, metric above threshold on a small fixed dataset.

```
pytest -q ; ruff check . ; black . ; pre-commit run --all-files
```

📺 https://www.youtube.com/results?search_query=TechWorld+with+Nana+GitHub+Actions+tutorial · https://www.youtube.com/results?search_query=CI+CD+for+machine+learning+GitHub+Actions

---

14. Databases

14.1 SQL & MySQL

🇬🇧 **Level 1:** A relational database stores data in tables linked by keys; SQL queries it. **Level 2:** Primary key, foreign key, normalization (1NF–3NF), indexes (B-tree speed up reads, slow writes), transactions with **ACID**, JOIN types, aggregation, window functions, CTEs. Your BookLingo schema (books, authors, languages, genres, ratings, comments, users with roles) is a textbook relational design: be ready to draw the ER diagram and explain 1-N and N-N (pivot table) relations.

🇫🇷 Une base relationnelle stocke des tables liées par des clés ; SQL l'interroge. Clé primaire/étrangère, normalisation, index (accélèrent la lecture, ralentissent l'écriture), transactions ACID, jointures, fonctions de fenêtrage.

```
SELECT a.name, COUNT(*) AS n_books, AVG(b.rating) AS avg_rating
FROM books b JOIN authors a ON a.id = b.author_id
WHERE b.language = 'fr'
GROUP BY a.name HAVING COUNT(*) >= 3
ORDER BY avg_rating DESC LIMIT 10;

-- window function: rolling 24h mean per station
SELECT ts, station, AVG(pm25) OVER (PARTITION BY station ORDER BY ts ROWS
BETWEEN 23 PRECEDING AND CURRENT ROW) AS pm25_24h
FROM measurements;
```

Q&A: *WHERE vs HAVING?* → WHERE filters rows before grouping, HAVING filters groups after. *INNER vs LEFT JOIN? What is an index and when does it hurt? Trap — what is the N+1 query problem in Eloquent?* → Loading relations in a loop issues one query per row; fix with eager loading (`with()`). *Where would you store IoT time series?* → A time-series DB (TimescaleDB/InfluxDB) or PostgreSQL partitioned by time rather than MySQL; know why.

📺 https://www.youtube.com/results?search_query=freeCodeCamp+SQL+full+course · https://www.youtube.com/results?search_query=SQL+window+functions+explained · https://www.youtube.com/results?search_query=DataTalksClub+SQL+for+data+engineering

14.2 Laravel stack (low priority for ML roles)

Know in 5 lines: **MVC** (Model–View–Controller), **Eloquent ORM** (maps tables to PHP classes), **migrations** (versioned schema changes), **Blade** (templating), **Breeze** (auth scaffolding), **middleware** (role-based access). 🇫🇷 Retenez : MVC, Eloquent (ORM),

migrations (schéma versionné), Blade (templates), Breeze (authentification), middleware (contrôle d'accès). Use it as proof you can ship a full product, not as a headline skill. 📺 https://www.youtube.com/results?search_query=Laravel+Breeze+tutorial

---

15. Architecture & System Design

🇬🇧 **Level 1:** System design means choosing components and how they talk, so the system meets goals such as latency, reliability and cost. **Level 2:** Vocabulary: latency vs throughput, p50/p95/p99, stateless services, horizontal vs vertical scaling, load balancer, queue/message broker, cache, idempotency, retries with backoff, single point of failure, observability (logs, metrics, traces), batch vs online inference, training/serving skew.

🇫🇷 La conception de système consiste à choisir composants et interactions pour respecter latence, fiabilité et coût. Vocabulaire : latence vs débit, services sans état, mise à l'échelle horizontale, file de messages, cache, idempotence, observabilité, décalage entraînement/service.

Decision	Option A	Option B	When
Inference	Online (API, ms)	Batch (nightly)	Real-time alerts vs reports
Ingestion	WebSocket	MQTT / Kafka	Few dashboards vs thousands of devices
Model format	Framework native	ONNX	Portability, lower latency
Storage	Local disk	Object storage (MinIO/S3)	Scalability, shared artifacts
Deploy	Compose on one VM	Kubernetes	Simple vs multi-node scale

Interview exercise: 'Design a real-time air-quality forecasting platform for 1,000 sensors.' Answer in order: requirements (latency, freshness, accuracy), data flow, ingestion (MQTT/Kafka), storage (time-series DB + object store), features, training pipeline + registry, serving (FastAPI/ONNX behind a load balancer), monitoring/drift, failure modes (sensor offline, model failure → fallback to persistence baseline), cost. **Training/serving skew** (features computed differently offline and online) is a classic trap: reuse the same preprocessing code in both.

📺 https://www.youtube.com/results?search_query=system+design+for+machine+learning+interview · https://www.youtube.com/results?

search_query=Designing+Machine+Learning+Systems+Chip+Huyen+explained

16. My Projects — Technical Deep Dive

For each project: what it really is, what an interviewer will probe, how to answer. **Rule:** every claim on the CV must have a number, a reason and a limitation you can state in 30 seconds.

16.1 AirSentinel (ENSA internship, Jul–Sep 2026) — your flagship

🇬🇧 **30-second pitch:** 'AirSentinel forecasts air quality 12 hours ahead from IoT sensor streams. Sensors push data through WebSocket endpoints into an asynchronous pipeline; a Transformer trained on historical data is exported to ONNX and served by a FastAPI service in Docker, answering in under a second; a Streamlit dashboard with PyDeck 3D maps and role-based access shows forecasts, even offline.'

🇫🇷 **Pitch de 30 secondes :** « AirSentinel prévoit la qualité de l'air à 12 heures à partir de flux de capteurs IoT. Les capteurs envoient leurs mesures via WebSocket vers un pipeline asynchrone ; un Transformer entraîné sur l'historique est exporté en ONNX et servi par FastAPI dans Docker, avec une réponse en moins d'une seconde ; un tableau de bord Streamlit avec cartes 3D PyDeck et accès par rôles affiche les prévisions, y compris hors ligne. »

Layer	Technology	Question to prepare
Data source	IoT sensors / dataset	Real sensors or replayed dataset? Which pollutants, sampling rate, how many stations?
Ingestion	WebSocket + asyncio	Message schema, validation, reconnection, backpressure, queue?
Storage	MinIO? files?	What is stored where, retention, formats (Parquet/CSV)?
Features	Windows, scaling	Window length, horizon, missing values, train/test split by time?
Model	Transformer	Architecture, parameters, loss, baselines, final RMSE/MAE by horizon?
Optimization	ONNX Runtime	Export tool, opset, quantization, latency before/after?
Serving	FastAPI in Docker	Startup loading, health endpoint, workers, tests?

Layer	Technology	Question to prepare
UI	Streamlit + PyDeck	Auth model, offline mode, caching, refresh rate?
Eval	'Environnement de test'	Who tested, which criteria, what feedback?

The 8 questions you will almost certainly get

1. What was the exact accuracy, and against which baseline (persistence, seasonal naive, RF)? *(Weak spot: the CV gives no accuracy number for your flagship project, although the other three projects do. Add one.)*
2. Why a Transformer and not LSTM/GBM? Did you compare?
3. How did you split data, and how did you avoid leakage?
4. How did you measure 'under one second'? Hardware? p95?
5. What happens if a sensor goes silent or sends garbage?
6. How would you retrain and redeploy a new model version without downtime?
7. How is the 3D map useful to a decision-maker?
8. What would you change if you had 10 000 sensors?

Ideal answer skeleton for any 'why' question: context → options considered → decision → evidence (number) → limitation.

16.2 BookLingo (Naja7Host, Jul–Aug 2025)

Full-stack proof: relational schema (Eloquent + migrations), auth with Breeze and role-based access, admin dashboards, moderation. Prepare: ER diagram, how roles are enforced (middleware/policies), how search by language/genre/author/rating is implemented (query + indexes), what moderation means technically. **Position it as:** 'it taught me the full lifecycle of a web product, which is why I deploy my ML models as real services.' Do not oversell it for ML roles.

16.3 Solar production forecasting (ARIMA)

See 5.2. Likely probes: units of RMSE, stationarity test, seasonal order, residual diagnostics (Ljung-Box, residuals look like white noise?), daylight handling, comparison with seasonal-naive and with ML models (RF with lags, Prophet, SARIMAX with weather exogenous variables). **Honest limitation:** ARIMA is univariate; solar output depends on irradiance/cloud cover, so exogenous variables are the obvious next step.

16.4 California housing (Random Forest)

The most common student dataset in the world: recruiters have seen hundreds of R^2 = 0.8x on it. It stays on the CV only if you can show depth: pipeline, CV, tuned hyperparameters, comparison with Linear/GBM, residual analysis, handling of the capped target (500k), geographic features (distance to coast/cities), SHAP/permutation importance. Otherwise shorten it to one line.

16.5 IoT sensor benchmark (MLP vs RF vs LSTM)

This is the project that shows scientific thinking: a fair comparison across models and variables. Prepare: what each sensor variable is, split strategy, lag design, per-variable winners, and the lesson 'no model wins everywhere; choose by signal dynamics and cost'. Ask yourself: is the winning margin larger than seed-to-seed variance?

---

17. Interview Questions (cross-cutting, rapid fire)

Format: Question → Concise answer → If they push.

Q1. Walk me through your ML project end-to-end. → Problem → data → split → baseline → model → metric → deployment → limits. *Push:* what did you try that failed?

Q2. How do you prevent data leakage? → Split first (chronologically for time series), fit preprocessing on train only (Pipeline), no future features, check suspiciously good scores. *Push:* give an example from your IoT project.

Q3. Accuracy is 99% — is the model good? → Depends on class balance/baseline; use precision/recall/F1/PR-AUC for imbalance. *(You have only regression on your CV; learn classification metrics too: confusion matrix, precision, recall, F1, ROC-AUC.)*

Q4. Bias vs variance? → Bias = too simple (underfit); variance = too sensitive to training data (overfit). Fix with regularization, more data, ensembling.

Q5. Explain gradient descent. → Iteratively move parameters opposite the gradient of the loss, step size = learning rate; mini-batches and Adam in practice. 📺 https://www.youtube.com/results?search_query=StatQuest+Gradient+Descent

Q6. How do you deploy a model? → Train → export (ONNX/pickle/joblib) → wrap in FastAPI → Docker image → run on server/cloud → monitor. *Push:* how do you roll back? → Keep versioned images/models, deploy by tag, route traffic gradually (canary/blue-green).

Q7. What is the difference between batch and real-time inference? → Batch: scheduled over many records, high throughput; real-time: single request, low latency.

Q8. What is REST? What is idempotency? → Resource-oriented HTTP API; an operation is idempotent if repeating it has the same effect as once (GET, PUT, DELETE yes; POST usually not).

Q9. Why Docker for ML? → Reproducible environment, same artifact from laptop to production, easy CI/CD.

Q10. How would you scale the prediction service? → Stateless containers → several replicas behind a load balancer; autoscale on CPU/latency; batch requests; use quantized model.

Q11. Overfitting in time series is subtle — why? → Random CV leaks the future; autocorrelation makes neighbours near-duplicates; use walk-forward validation.

Q12. SQL: top 3 products per category. → `ROW_NUMBER() OVER (PARTITION BY category ORDER BY sales DESC)` then filter `<= 3`.

Q13. Trap: 'You list MLOps on your CV. What is MLOps?' → Use the lifecycle diagram (11) and state honestly that your project covers packaging/serving and that you are adding tracking and CI/CD. Honesty plus a plan beats bluffing.

Q14. Trap: 'What is the difference between ONNX and a pickle file?' → Pickle is Python-specific and unsafe to load from untrusted sources; ONNX is language-agnostic and optimized for inference.

Q15. Trap: 'How do you handle a model whose latency is too high?' → Profile; reduce input/model size; quantization; ONNX graph optimization; batching; GPU/accelerators; caching; asynchronous precompute (forecast every 10 min and serve cached results).

Behavioural (PFE): Tell me about a technical problem you solved → use STAR (Situation, Task, Action, Result) with numbers. Why this company? Why ML engineering rather than networks? → 'Networks and IoT gave me the data sources and systems thinking; ML turns that data into decisions.' *(This is a strong, true differentiator — use it.)* 🇫🇷 Pourquoi le ML plutôt que les réseaux ? « Les réseaux et l'IoT m'ont donné les sources de données et la vision système ; le ML transforme ces données en décisions. »

---

18. CV Weaknesses (strict recruiter mode)

What makes your CV strong?

- A coherent story: IoT/time series/forecasting/deployment.
- Quantified results with baselines (23 % over persistence is the right instinct).
- Production-oriented stack in one project: ONNX + FastAPI + Docker + WebSockets + dashboard.

- Certificates (IBM, Oracle OCI) that align with the target role.
- Engineering school with a Networks/Telecom angle — rare among data candidates.
- Clear, targeted header: PFE, 6 months, January 2027, national mobility.

What makes it look junior?

- Three of your projects are classic academic datasets/problems (housing, solar ARIMA, sensor benchmark) with notebook-level stacks (no tests, no API, no repo structure mentioned).
- No link to a deployed demo or a GitHub repo per project, no README/architecture diagram.
- No metrics for the flagship project.
- No mention of teamwork on a real data team, code review, agile, or an external company ML experience.

Which skills look superficial?

- **MinIO**: listed in skills, absent from every bullet.
- **'Pipelines de données'**: vague; which ones?
- **'MLOps et déploiement'** heading: you show serving, not MLOps.
- **Transformer**: is it your own implementation, a library model, a pretrained one? Specify.
- **SQL**: listed first among languages but no project uses SQL analytics (only MySQL via Laravel). Add one analytical SQL piece.
- **Java, PHP** and **Oracle certification**: fine as support, weak as headlines.
- Language levels: 'Français (avancé)' invites a French interview; make sure the level you write is the level you can sustain (if your tested level is B2, write 'courant (B2)' rather than risk looking inflated).

Which technologies are missing?

MLflow/experiment tracking, DVC, GitHub Actions/CI, pytest/testing, Linux/Bash, PostgreSQL, a gradient boosting library (XGBoost/LightGBM), PyTorch, a cloud deployment, monitoring (Prometheus/Grafana/Evidently), orchestration (Airflow/Prefect), message brokers (MQTT/Kafka) — and, for classification roles, any classification project at all. Also absent: SHAP/explainability, Hugging Face, and any NLP/CV, which is fine as long as you target time-series/IoT roles.

Which projects are impressive?

AirSentinel (once quantified) and the IoT benchmark (scientific comparison).

Which look like basic academic projects?

California housing (very common) and, to a lesser extent, solar ARIMA, unless you add depth and links.

Keywords missing for DS / ML Engineer / MLOps screening (ATS)

'Model evaluation', 'cross-validation', 'hyperparameter tuning', 'XGBoost', 'PyTorch', 'MLflow', 'CI/CD', 'GitHub Actions', 'pytest', 'REST API', 'model monitoring', 'data drift', 'Linux', 'PostgreSQL', 'Kafka/MQTT', 'Airflow', 'Prometheus/Grafana', 'cloud (OCI/AWS)', 'anomaly detection', 'reproducibility'. Add only those you can defend.

What would make a recruiter reject the CV?

- No visible GitHub/portfolio evidence next to impressive claims (check that your GitHub link on the CV works and matches your actual profile URL — a broken or mismatched link is a silent rejection).
- Job title inflation: 'Ingénieure ML' for a school internship can look inflated to a recruiter. Safer: 'Stagiaire ingénieure ML' with the context (who supervised, what was delivered).
- Skills list longer than evidence.
- Not enough professional (company) experience; mitigate with strong, linked projects.
- A CV in French only if the company is international; keep a one-page English version.

What would make a technical interviewer doubt your claims?

- Not knowing the real number for AirSentinel.
- Not being able to explain how ONNX export or the 'under one second' was achieved.
- Not knowing what MinIO is used for in your stack.
- Explaining ARIMA without mentioning stationarity.
- Not knowing the difference between async and parallel.

---

19. Skills I Should Learn Next (priority order)

1. **Testing + Git hygiene + Linux** — cheap, high signal (2 weeks).
2. **MLflow + DVC** on one of your existing projects (2 weeks).
3. **GitHub Actions** CI with tests + Docker build (1 week).
4. **XGBoost/LightGBM + SHAP** (1 week) — modern tabular default.
5. **PostgreSQL + advanced SQL** (2 weeks).
6. **Monitoring** (Prometheus/Grafana or Evidently) on AirSentinel (2 weeks).

7. **Cloud deployment** of one container (OCI or AWS free tier) (1 week).
8. **PyTorch basics** (2–3 weeks).
9. Later: Airflow/Prefect, Kafka/MQTT, Kubernetes concepts.

🇫🇷 **À apprendre en priorité :** tests + Git + Linux ; MLflow + DVC ; GitHub Actions ; XGBoost/SHAP ; PostgreSQL ; monitoring ; déploiement cloud ; bases de PyTorch ; puis Airflow, Kafka/MQTT, Kubernetes.

---

20. High-Impact Portfolio Projects

Chosen to extend what you already do (IoT, time series, serving) so each project adds *proof* rather than new buzzwords. Pick **two**, finish them properly (README, architecture diagram, tests, demo video), rather than starting all four.

Project A — AirSentinel v2: Production MLOps for Air-Quality Forecasting *(highest ROI — upgrade what you have)*

- **Business problem:** cities/campuses need reliable, monitored 12-hour pollution forecasts and alerts; models degrade as sensors and seasons change.
- **Why impressive:** the same project the recruiter already sees, now with the missing production layer.
- **Architecture:** `Sensors/replay -> MQTT (Mosquitto) or WebSocket -> ingestion service -> PostgreSQL/TimescaleDB + MinIO -> feature job -> training (PyTorch/Keras) -> MLflow (tracking + registry, artifacts in MinIO) -> ONNX export -> FastAPI (Docker) -> Streamlit -> Prometheus + Grafana -> drift alert -> retrain`.
- **Technologies:** Python, PostgreSQL/TimescaleDB, MQTT, MinIO, MLflow, DVC, FastAPI, ONNX Runtime, Docker Compose, GitHub Actions, Prometheus, Grafana, Evidently.
- **Dataset/source:** open air-quality data (OpenAQ, UCI Air Quality, national agencies) plus your own IoT data; replay script to simulate streams.
- **ML component:** Transformer vs LSTM vs LightGBM vs seasonal-naive baseline, walk-forward validation, prediction intervals (quantile loss), anomaly detection for sensor faults.
- **Data engineering component:** idempotent ingestion, schema validation, resampling/gap filling, partitioned storage.
- **MLOps component:** tracked experiments, model registry with champion alias, data versioning, automated retraining job on drift.
- **Deployment:** Compose on a VM (OCI free tier or any VPS); optional k3s later.

- **Monitoring:** API latency p95, error rate, rolling forecast error against realized values, PSI drift on inputs.
- **CI/CD:** lint + tests + image build + push + deploy on merge to main, smoke test of `/predict`.
- **CV bullet:** *Built a production-grade air-quality forecasting platform: MQTT ingestion, MLflow-tracked Transformer/LightGBM benchmark (RMSE X vs baseline Y), ONNX serving at p95 < Z ms, Dockerized with CI/CD and Grafana drift monitoring.*
- **Interview asks:** how to detect drift; why retrain triggers; model rollback; MQTT vs WebSocket; how registry promotion works.
- **Difficulty:** medium–high. **Time:** 6–8 weeks part-time.

Project B — Predictive Maintenance Platform for Telecom / Network Equipment

- **Business problem:** predict failures (overheating, link degradation) of network devices before outages — uses your Networks & Telecom edge.
- **Why impressive:** unique intersection of telecom + ML; few candidates have it.
- **Architecture:** `SNMP/syslog-like telemetry (simulated or from NASA C-MAPSS / Backblaze drive data) -> Kafka or Redpanda -> stream processing -> feature store (Parquet/Postgres) -> LightGBM/LSTM RUL model -> MLflow -> FastAPI -> Docker Compose -> Grafana alerts`.
- **Technologies:** Python, Kafka (or Redpanda), PostgreSQL, MLflow, FastAPI, Docker Compose, Prometheus/Grafana, GitHub Actions.
- **Dataset:** NASA C-MAPSS turbofan (Remaining Useful Life), Backblaze open hard-drive stats, plus a simulator generating network KPIs.
- **ML component:** survival/RUL regression and failure classification with class imbalance (PR-AUC, recall at fixed precision), time-aware split.
- **Data engineering component:** event-time processing, late data, windowed aggregations.
- **MLOps/Deployment/Monitoring:** same pattern as A, plus alert thresholds and cost-sensitive decisions (false alarm vs missed failure).
- **CV bullet:** *Designed a streaming predictive-maintenance pipeline (Kafka, LightGBM, MLflow, FastAPI, Grafana) reaching PR-AUC X on failure prediction with Y h lead time.*
- **Interview asks:** class imbalance, leakage in sliding windows, precision/recall trade-off in operations, Kafka partitions/consumer groups.
- **Difficulty:** high. **Time:** 8–10 weeks.

Project C — Data Engineering Pipeline: Open Data Lakehouse with Orchestration and Quality Checks

- **Business problem:** turn raw public datasets (weather + air quality + traffic) into trusted analytics tables.
- **Why impressive:** shows data engineering craft rarely seen in student ML CVs.
- **Architecture:** `APIs/CSV -> Airflow DAG -> raw zone (MinIO/Parquet) -> dbt or SQL transforms -> PostgreSQL warehouse -> Great Expectations tests -> BI dashboard (Metabase/Superset)`.
- **Technologies:** Airflow (or Prefect), MinIO, Parquet, PostgreSQL, dbt, Great Expectations, Docker Compose, GitHub Actions.
- **Dataset:** OpenAQ + Open-Meteo weather + public traffic data.
- **ML component:** a downstream forecasting model reading from the warehouse.
- **Data engineering component:** incremental loads, idempotent DAGs, schema evolution, data contracts, backfills.
- **MLOps/Deployment:** containerized Airflow; CI runs dbt tests; documentation of lineage.
- **Monitoring:** DAG failure alerts, freshness checks, data-quality dashboards.
- **CV bullet:** *Built an orchestrated ELT pipeline (Airflow, MinIO/Parquet, PostgreSQL, dbt, Great Expectations) ingesting 3 open APIs daily with automated data-quality gates.*
- **Interview asks:** ETL vs ELT, idempotency, partitioning, why Parquet, backfills, SCD.
- **Difficulty:** medium. **Time:** 4–6 weeks.

Project D — Anomaly Detection & Explainability for IoT Sensors (edge-aware)

- **Business problem:** detect faulty sensors or abnormal pollution events and explain why.
- **Why impressive:** unsupervised/semi-supervised ML + explainability + edge deployment: advanced but accessible.
- **Architecture:** `sensor stream -> windowing -> Isolation Forest / autoencoder / residual-based detector on forecast errors -> SHAP explanations -> FastAPI -> alert service -> dashboard; ONNX/quantized model benchmarked on Raspberry Pi/CPU`.
- **Technologies:** Python, PyTorch/Keras, scikit-learn, SHAP, ONNX Runtime, FastAPI, Docker, MLflow.
- **Dataset:** NAB (Numenta Anomaly Benchmark), SKAB, your IoT data with injected faults.
- **ML component:** anomaly detection metrics (precision/recall with tolerance windows), threshold selection.
- **Data engineering component:** streaming windows, state handling.

- **MLOps/Deployment/Monitoring/CI/CD:** versioned detector, latency/memory benchmark on CPU, tests with synthetic anomalies.
- **CV bullet:** *Developed an explainable IoT anomaly-detection service (Isolation Forest + autoencoder, SHAP) served with ONNX Runtime at < N ms on CPU, evaluated on NAB/SKAB.*
- **Interview asks:** how to evaluate without labels, threshold choice, concept drift, why explainability matters to operators.
- **Difficulty:** medium–high. **Time:** 4–6 weeks.

Recommendation: A first (fastest credibility gain, reuses your work), then B *or* C depending on whether you target ML Engineer/MLOps (B) or Data Engineer (C). D is the optional differentiator.

---

21. Skills That Would Upgrade My CV

Format per skill: **Why it matters → What to learn → How to prove it → How it appears on CV**.

🔥 Highest ROI

Skill	Why	Learn	Prove	CV line
Git workflow + pytest	Basic engineering credibility	branches, PRs, fixtures, API tests	tests + CI badge in repo	'Unit/API tests with pytest, CI via GitHub Actions'
MLflow	Standard experiment tracking	runs, registry, artifacts	AirSentinel v2	'Experiment tracking and model registry with MLflow'
GitHub Actions	Shows automation	workflows, secrets, Docker build	CI on all repos	'CI/CD pipeline (lint, test, build, push)'
Linux/Bash	Servers & containers	CLI, permissions, ssh	deploy on a VM by hand	'Linux (Ubuntu), Bash scripting'

Skill	Why	Learn	Prove	CV line
XGBoost/LightGBM + SHAP	Tabular state of the art	boosting, tuning, explainability	replace housing RF	'Gradient boosting (LightGBM), SHAP explainability'

🚀 Strong differentiators

Skill	Why	Learn	Prove	CV line
IoT/MQTT + edge inference	Your telecom background	MQTT QoS, topics, Mosquitto, ONNX on CPU/Pi	Project A or D	'MQTT ingestion, edge inference benchmarking'
Time-series depth	Your niche	SARIMAX, Prophet, walk-forward CV, prediction intervals	benchmark with intervals	'Probabilistic forecasting, walk-forward validation'
Model optimization	Latency/cost	quantization, graph optimization	p95 before/after table	'ONNX quantization: -X% latency'

☁️ Cloud / MLOps

Skill	Why	Learn	Prove	CV line
One cloud deploy (OCI or AWS)	Needed for most roles	VM, object storage, IAM, security groups	public demo URL	'Deployed containerized ML API on OCI/AWS'
Prometheus + Grafana	Monitoring	metrics, exporters, dashboards	latency & drift dashboard	'Monitoring with Prometheus/Grafana'
DVC	Reproducibility	remotes, pipelines	`dvc repro` in repo	'Data versioning with DVC'

Skill	Why	Learn	Prove	CV line
Kubernetes (basics)	Frequent in job posts	pods, deployments, services (k3s/minikube)	deploy the API locally	'Kubernetes (k3s) basics: deployment, service' — only if built

📊 Data Engineering

Skill	Why	Learn	Prove	CV line
PostgreSQL + advanced SQL	Core of DE/DS	window functions, indexes, EXPLAIN	Project C	'PostgreSQL, advanced SQL (window functions, CTEs)'
Airflow/Prefect	Orchestration	DAGs, retries, schedules	Project C	'Workflow orchestration with Airflow'
Parquet + dbt	Modern analytics stack	columnar formats, models, tests	Project C	'dbt, Parquet'
Kafka (concepts + one demo)	Streaming	topics, partitions, consumer groups	Project B	'Kafka streaming pipeline'

🧠 Advanced ML

Skill	Why	Learn	Prove	CV line
PyTorch	Industry/research standard	tensors, autograd, training loop	rewrite Transformer	'PyTorch'
Explainability + uncertainty	Trust in decisions	SHAP, quantile/conformal intervals	forecasts with intervals	'Explainability (SHAP), uncertainty quantification'
Anomaly detection	Real IoT need	Isolation Forest, autoencoder	Project D	'Anomaly detection (NAB/SKAB)'

Skill	Why	Learn	Prove	CV line
Classification & imbalance	Missing on CV	F1, PR-AUC, resampling	Project B	'Imbalanced classification (PR-AUC)'

> Rule: add a skill to the CV only after you have a repo proving it and can answer three interview questions about it.

---

22. 3 / 6 / 12-Month Learning Roadmap

Adapted to your situation: PFE start **January 2027** (about 2.5 months away) and applications happening now.

Now → 3 months (until PFE start, Oct 2026 – Jan 2027): *Make the CV defensible*

- **Weeks 1–2:** fix CV (quantify AirSentinel, clarify titles, remove or justify MinIO, GitHub links, French level). Write STAR stories for each project. Practice the two 30-second pitches in EN and FR.
- **Weeks 2–4:** Git + pytest + Linux basics; add tests to AirSentinel's API; add README + architecture diagram.
- **Weeks 4–6:** add MLflow + a LightGBM baseline + seasonal-naive baseline to the IoT benchmark; log everything.
- **Weeks 6–9:** GitHub Actions CI + Docker Compose with MinIO + a Grafana latency dashboard.
- **Weeks 9–12:** SQL/PostgreSQL practice (50 exercises), classification metrics, mock interviews (one per week, record yourself).
- **Milestone:** Project A partially live; 10 core topics defensible; CV v2 sent to targeted companies.

3 → 6 months (during the PFE, Jan – Jun 2027): *Make it industrial*

- Ship real work at the company: document architecture, tests, deployment, monitoring — this becomes your strongest CV section.
- Learn on the job: orchestration (Airflow/Prefect), cloud deployment, Kubernetes concepts depending on the company stack.
- Side project: Project C or D (4–6 weeks, weekends).
- Read: *Designing Machine Learning Systems* (Chip Huyen) alongside; follow DataTalksClub MLOps Zoomcamp at a pace of 2–3 modules/month.

6 → 12 months (after the PFE, Jul – Dec 2027): *Specialize*

- Choose a lane: **ML Engineer / MLOps** (K8s, cloud, monitoring, feature stores), **Data Scientist** (probabilistic forecasting, causal inference, experimentation), or **Data Engineer** (Spark, Kafka, dbt, lakehouse).
- Optional certification matching the lane (AWS ML Specialty, CKA/CKAD, Databricks, dbt), but only after the project.
- Contribute to an open-source tool you used (MLflow, Evidently, FastAPI docs): visible proof of depth.
- Target: one flagship, production-style repo with documentation and a blog post per quarter.

🇫🇷 **Résumé de la feuille de route :** 0–3 mois = rendre le CV défendable (tests, MLflow, CI, SQL, entraînement aux entretiens) ; 3–6 mois = industrialiser pendant le PFE (déploiement, monitoring, orchestration) ; 6–12 mois = se spécialiser (ML Engineer/MLOps, Data Scientist ou Data Engineer) et publier.

---

23. Final Interview Survival Sheet

Format: **Technology → What → Why → Real example → Question → Answer.** (🇬🇧 then 🇫🇷)

Python — 🇬🇧 General-purpose language for data/ML. Why: huge ecosystem. Example: whole AirSentinel stack. Q: What is the GIL? A: A lock letting one thread run Python bytecode at a time; use multiprocessing or native libraries for CPU work. 🇫🇷 Langage généraliste pour la data/ML. Q : Qu'est-ce que le GIL ? R : Un verrou qui n'autorise qu'un thread à exécuter du bytecode Python à la fois.

Pandas/NumPy — 🇬🇧 Tabular and numeric computing. Why: fast vectorized operations. Example: `resample`, `shift` for lags. Q: Why vectorize? A: Loops run in Python; vectorized ops run in C. 🇫🇷 Calcul tabulaire et numérique ; vectorisation = exécution en C, bien plus rapide.

Scikit-learn — 🇬🇧 Classic ML library. Why: consistent fit/predict API. Example: Pipeline with imputer + scaler + RF. Q: Why a Pipeline? A: Same preprocessing in train and serve; prevents leakage in CV. 🇫🇷 Bibliothèque ML classique ; Pipeline = même prétraitement partout, pas de fuite.

Time series / ARIMA — 🇬🇧 Forecasting ordered data. Why: demand, energy, sensors. Example: solar ARIMA beat persistence by 23 %. Q: What is stationarity? A: Constant mean/variance/autocorrelation over time; ARIMA differences the series to reach it. 🇫🇷 Prévision de données ordonnées ; stationnarité = statistiques constantes dans le temps.

Random Forest — 🇬🇧 Ensemble of decorrelated trees. Why: strong tabular baseline, little tuning. Example: housing R^2 0.85, humidity benchmark. Q: Why does it overfit less than one tree? A: Averaging many de-correlated trees reduces variance. 🇫🇷 Ensemble d'arbres décorrélés ; la moyenne réduit la variance.

MLP / LSTM — 🇬🇧 Feed-forward network / gated recurrent network. Why: nonlinear patterns / sequences. Example: IoT benchmark. Q: What problem do LSTM gates solve? A: Vanishing gradients, by controlling what is remembered or forgotten. 🇫🇷 Réseau dense / réseau récurrent à portes ; résout le gradient qui disparaît.

Transformer — 🇬🇧 Attention-based sequence model. Why: parallel training, long-range dependencies. Example: 12 h air-quality forecast. Q: What does self-attention compute? A: Weights over all positions via $\text{softmax}(QK^T/\sqrt{d})$, then a weighted sum of values. 🇫🇷 Modèle à attention ; l'auto-attention pondère toutes les positions.

ONNX Runtime — 🇬🇧 Cross-framework model format + fast inference engine. Why: portability, latency. Example: Transformer served in < 1 s. Q: How do you check an export? A: Compare outputs with the original model on the same inputs with a tolerance. 🇫🇷 Format de modèle portable + moteur d'inférence rapide ; on valide l'export en comparant les sorties.

FastAPI — 🇬🇧 Typed Python web framework. Why: speed, validation, auto docs. Example: `/predict`, `/health`, WebSocket ingest. Q: Why load the model at startup? A: Loading is slow; do it once and reuse. 🇫🇷 Framework web typé ; on charge le modèle au démarrage pour éviter un coût à chaque requête.

WebSocket — 🇬🇧 Persistent two-way connection. Why: low-latency streaming. Example: sensor ingestion. Q: WebSocket vs polling? A: One connection and server push vs repeated requests. 🇫🇷 Connexion bidirectionnelle persistante, évite le polling.

Docker / Compose — 🇬🇧 Containers / multi-container definition. Why: reproducibility. Example: api + dashboard + minio. Q: Image vs container? A: Image is the immutable template; container is a running instance. 🇫🇷 Image = modèle immuable ; conteneur = instance en cours d'exécution.

MinIO — 🇬🇧 S3-compatible object storage. Why: store datasets and model artifacts. Example: ONNX versions per bucket. Q: Object storage vs file system? A: Flat key-based objects over HTTP, scalable, no folders. 🇫🇷 Stockage objet compatible S3, accessible par HTTP.

Streamlit/PyDeck — 🇬🇧 Python dashboards / geospatial layers. Why: fast prototyping. Example: 3D pollution map. Q: Why `session_state`? A: Streamlit reruns the script on each interaction. 🇫🇷 Tableaux de bord en Python ; `session_state` conserve l'état entre relances.

SQL/MySQL — 🇬🇧 Relational querying. Why: data lives in databases. Example: BookLingo schema. Q: WHERE vs HAVING? A: Row filter before grouping vs group filter after. 🇫🇷 Interrogation relationnelle ; WHERE filtre les lignes, HAVING filtre les groupes.

Git/GitHub — 🇬🇧 Version control. Why: history, collaboration. Example: feature branches + PRs. Q: Merge vs rebase? A: Merge keeps history with a merge commit; rebase rewrites commits onto a new base for linear history. 🇫🇷 Gestion de versions ; merge conserve l'historique, rebase le linéarise.

MLflow / DVC / CI-CD / Monitoring (to add) — 🇬🇧 Track experiments / version data / automate tests and delivery / watch production. Why: reproducibility and reliability. Q: What is data drift? A: A change in the input distribution between training and production. 🇫🇷 Suivre les expériences / versionner les données / automatiser tests et livraison / surveiller la production ; la dérive des données = changement de distribution des entrées.

Oracle OCI Data Science (certificate) — 🇬🇧 Oracle's managed data-science platform. Why: notebooks, jobs, model deployment in the cloud. Q: What did you actually build with it? A: *(prepare a real lab/mini-project answer.)*

Last honest advice

Your profile is already coherent. The gap between 'good student CV' and 'candidate they call first' is **proof**: numbers on the flagship project, tests, CI, tracking, one monitored deployment, and a clean GitHub. Fix the 10 items in section 18 first; learn in the order of section 19; then build Project A.